YENEPOYA UNIVERSITY

LOW LEVEL DESIGN (LLD) DOCUMENT
ON
Cloud Cost Intelligence For Smart City

Student Name: Shuhaib Abdulla

Registration No: 23BCAICD101

Campus ID: 25296

Project Code: PRJN26-108

Industry Mentor: Sumit Kumar Shukla

Guided By: Ms. Deeksha Shetty

TABLE OF CONTENTS

1. Introduction
   1.1 Document Purpose
   1.2 Intended Audience
2. LLD Scope and Execution Pipeline
   2.1 Pipeline Stages
   2.2 Execution Flow Diagram
3. Input / Output Data Structures
   3.1 Input CSV Schema
   3.2 Intermediate Data Structures
   3.3 Output Artefacts
4. Module-Level Logic
   4.1 Data Ingestion Module
   4.2 Aggregation Engine
   4.3 Pivot and Temporal Analysis
   4.4 Visualisation Module
   4.5 Export Module
5. Function Specifications
   5.1 Core Function Contracts
   5.2 Aggregation Logic
   5.3 Chart Generation Logic
6. Error Handling
   6.1 Input Validation
   6.2 Runtime Guards
7. Configuration and Constants
8. Conclusion

1. INTRODUCTION

1.1 Document Purpose

TheL ow-LevelDesign (LLD) document serves as the definitive technical blueprint for the internal mechanics of the Cloud Cost Intelligence Platform for Smart City (PRJN26-108). While the High-Level Design (HLD) outlines the general system architecture and component interactions, this document dives into the granular, programmatic specifics.

It defines the exact input/output data contracts, the step-by-step transformation logic applied to the billing DataFrame, the precise function-level specifications for aggregation and charting operations, error handling strategies, and the configuration constants that govern pipeline behaviour. This document is designed to guide the development, testing, and replication phases by eliminating architectural ambiguity.

1.2 Intended Audience

- **Project Reviewers and Evaluators:** To demonstrate the rigorous technical logic and step-by-step data engineering pipeline underlying the platform for comprehensive academic assessment.
- **Developers and Data Engineers:** To provide precise programmatic specifications for building each pipeline module — ingestion, aggregation, visualisation, and export.
- **Data Scientists and Analysts:** To document the exact groupby logic, rolling statistics, pivot structure, and chart formatting conventions used throughout the analysis.
- **Future Maintainers:** To serve as a definitive reference guide for extending the platform to support additional data sources, visualisation types, or deployment environments.

2. LLD SCOPE AND EXECUTION PIPELINE

This section details the internal execution mechanics of the Cloud Cost Intelligence Platform. The system operates as a linear, deterministic pipeline — each stage consumes the output of the previous stage and produces a well-defined artefact consumed by the next stage.

2.1 Pipeline Stages

Stage	Module	Input	Output
1 – Ingestion	CSV Loader	smart_city_billing.csv	Master DataFrame df
2 – Exploration	EDA Logger	DataFrame df	Console log (shape, dtypes, stats, nulls)
3 – Service Agg.	Groupby Engine	df[Service_Type, Cost_USD]	Series: service_cost (sorted desc)
4 – Monthly Agg.	Groupby Engine	df[Month, Cost_USD]	Series: monthly_cost (calendar-ordered)
5 – Dept. Agg.	Groupby Engine	df[Department, Cost_USD]	Series: dept_cost (sorted asc)
6 – Pivot	Pivot Engine	df[Month, Service_Type, Cost_USD]	DataFrame: pivot (Month × Service_Type)
7 – Charts	Matplotlib Engine	service_cost, monthly_cost, pivot, dept_cost	4 × PNG chart files
8 – Export	OpenPyXL Writer	service_cost, monthly_cost, dept_cost	Excel workbook (.xlsx)

2.2 Execution Flow Summary

The notebook executes sequentially, with each cell building on prior results. Global configuration (DPI settings, float formatting, warning suppression) is applied once at Cell 1. The master DataFrame is created at Cell 4 and shared across all subsequent cells. No cell should be skipped, as downstream cells depend on variables created in prior cells.

3. INPUT / OUTPUT DATA STRUCTURES

3.1 Input CSV Schema

Thes ystem acceptsa single CSV file as its sole external input. The file must conform to the following schema. Any deviation will raise a KeyError during the groupby operations.

Column Name	Dtype	Constraints	Example Value
Invoice_ID	str	Non-null, unique	INV-001
Month	str	Must be full month name (January…December)	January
Service_Type	str	One of: Compute, Storage, Networking, Database, AI/ML Services	Compute
Department	str	One of: Traffic Management, Public Safety, Smart Lighting, Waste Management, Water Supply	Traffic Management
Usage_Hours	int	Non-null, ≥0	720
Cost_USD	float	Non-null, ≥0.0	1440.00

3.2 Intermediate Data Structures

Variable	Type	Shape	Description
df	pd.DataFrame	(60, 6)	Master billing DataFrame loaded from CSV
service_cost	pd.Series	(5,)	Total Cost_USD grouped by Service_Type, sorted descending
monthly_cost	pd.Series	(12,)	Total Cost_USD grouped by Month, reindexed to calendar order
dept_cost	pd.Series	(5,)	Total Cost_USD grouped by Department, sorted ascending
pivot	pd.DataFrame	(12, 5)	Month × Service_Type cost pivot table, reindexed to calendar order

3.3 Output Artefacts

Artefact	Format	Filename	Contents
Service Pie Chart	PNG (DPI 120)	pie_by_service.png	Exploded pie showing % and $ share per service type
Monthly Pie Chart	PNG (DPI 120)	pie_by_month.png	Pie showing monthly $ cost and % share for Jan–Dec
Stacked Bar Chart	PNG (DPI 120)	bar_monthly_stacked.png	Stacked bar of monthly spend broken down by service type
Dept. Bar Chart	PNG (DPI 120)	bar_by_department.png	Horizontal bar of total spend per smart city department
Excel Report	XLSX	cloud_cost_report.xlsx	Multi-sheet workbook: Service Summary, Monthly Summary, Dept Summary

4. MODULE-LEVEL LOGIC

4.1 Data Ingestion Module

The ingestionm oduleperformsthe following exact operations in order:

- Import all required libraries: **pandas**, **matplotlib.pyplot**, **matplotlib.patches**, **warnings**, **openpyxl**.
- Suppress all runtime warnings via **warnings.filterwarnings('ignore')** to prevent non-critical deprecation messages from cluttering notebook output.
- Configure float display precision to 2 decimal places: **pd.set_option('display.float_format', '{:.2f}'.format)**.
- Set chart render resolution: **plt.rcParams['figure.dpi'] = 100**.
- Load the CSV: **df = pd.read_csv('smart_city_billing.csv')**. The resulting DataFrame must have shape (60, 6).
- Validate load success by printing total record count, column list, and first 10 rows.

4.2 Aggregation Engine

Threei ndependentgroupbyaggregations are computed from the master DataFrame:

Service-Type Aggregation:

```
service_cost = df.groupby('Service_Type')['Cost_USD'].sum()
.sort_values(ascending=False)
```

Groups all 60 records by Service_Type, sums Cost_USD within each group, and sorts descending so the highest-cost service appears first in charts.

Monthly Aggregation:

```
month_order = ['January', 'February', ..., 'December']
monthly_cost = df.groupby('Month')['Cost_USD'].sum()
.reindex(month_order)
```

Groups records by Month and sums Cost_USD. The critical step is **reindex(month_order)** — without this, months are returned in alphabetical order (April, August, December…), which is meaningless for temporal analysis. The reindex ensures calendar-correct ordering in all downstream charts.

Departmental Aggregation:

```
dept_cost = df.groupby('Department')['Cost_USD'].sum()
.sort_values(ascending=True)
```

Groups records by Department and sums Cost_USD, sorted ascending so the lowest-spending department appears at the bottom of the horizontal bar chart — which reads naturally from bottom to top in ascending cost order.

4.3 Pivot and Multi-Dimensional Analysis

```
pivot = df.pivot_table(values='Cost_USD',
index='Month',
columns='Service_Type',
aggfunc='sum')
pivot = pivot.reindex(month_order)
```

Constructs a 12 × 5 pivot table where rows represent months (calendar-ordered) and columns represent the five service types. Each cell contains the total Cost_USD for that month-service combination. This structure is the direct input to the stacked bar chart and enables multi-dimensional spend analysis in a single DataFrame operation.

4.4 Visualisation Module

The visualisation module produces four charts using the following rendering logic:

Chart	Function Call	Key Parameters
Service Pie	ax.pie(service_cost.values, labels=..., autopct=...)	explode=[0.05]*5, shadow=True, startangle=140, colors=5-colour palette
Monthly Pie	ax.pie(monthly_cost.values, autopct=lambda p: ...)	Custom lambda autopct returning both $ value and % — requires monthly_cost.sum() in closure
Stacked Bar	pivot.plot(kind='bar', stacked=True, ...)	figsize=(12,7), edgecolor='white', legend bbox_to_anchor=(1.01,1)
Dept. Bar	ax.barh(dept_cost.index, dept_cost.values, ...)	figsize=(10,5), value labels added via ax.text() at bar.get_width()+30

All charts are saved via **plt.savefig(filename, bbox_inches='tight', dpi=120)** before **plt.show()** is called. The **bbox_inches='tight'** parameter is critical for the stacked bar chart, which has a legend positioned outside the axes frame — without this parameter, the legend is clipped from the saved file.

4.5 Export Module

The Excel export module uses **pd.ExcelWriter** with the **openpyxl** engine to write three summary sheets into a single workbook.

```
with pd.ExcelWriter('cloud_cost_report.xlsx', engine='openpyxl') as writer:
service_df.to_excel(writer, sheet_name='Service Summary', index=True)
monthly_df.to_excel(writer, sheet_name='Monthly Summary', index=True)
dept_df.to_excel(writer, sheet_name='Department Summary', index=True)
```

Each Series is converted to a DataFrame before writing. The workbook is finalised and flushed to disk when the context manager exits. The **index=True** parameter preserves the groupby key (service name, month name, or department name) as a labelled column in each sheet.

5. FUNCTION SPECIFICATIONS

5.1 Core Function Contracts

Function / Method	Input	Output	Side Effects
pd.read_csv(path)	File path string	pd.DataFrame (60, 6)	None — pure read
df.groupby(col)[col2].sum()	Column name strings	pd.Series (indexed by group key)	None — pure transform
series.reindex(list)	Ordered list of index labels	pd.Series (reordered)	None — pure transform
df.pivot_table(...)	Column name strings + aggfunc	pd.DataFrame (Month × Service_Type)	None — pure transform
plt.savefig(path, ...)	File path + kwargs	None	Writes PNG file to disk
pd.ExcelWriter(path)	File path + engine	Writer context manager	Writes XLSX on context exit

5.2 Aggregation Logic — Detailed Formulas

The corea ggregationformulasappliedtothebillingDataFrame are:

Total Cost per Service Type:

```
service_cost[s] = Σ Cost_USD ∀ rows where Service_Type = s
```

Total Cost per Month:

```
monthly_cost[m] = Σ Cost_USD ∀ rows where Month = m
```

Total Cost per Department:

```
dept_cost[d] = Σ Cost_USD ∀ rows where Department = d
```

Monthly-Service Pivot Cell:

```
pivot[m][s] = Σ Cost_USD ∀ rows where Month = m AND Service_Type = s
```

Monthly Cost Percentage (Pie Label):

```
label(m) = f"${p × monthly_cost.sum() / 100:,.0f}\n({p:.1f}%)"
where p = monthly_cost[m] / monthly_cost.sum() × 100
```

5.3 Chart Generation Logic — Key Technical Decisions

- **Explodeparameter(Pie Chart):**explode=[0.05] * len(service_cost) — dynamically sized to match the number of service categories, so the list length never mismatches the data.

- **Lambda autopct (Monthly Pie):** The lambda captures monthly_cost.sum() in its closure to convert the percentage argument p back to an absolute dollar value for display.
- **Bar total annotations:** For the stacked bar chart, per-bar total labels are added by iterating `enumerate(pivot.iterrows())` and computing `row.sum()` for each month.
- **x-axis limit (Dept. Bar):** ax.set_xlim(0, dept_cost.max() * 1.2) — adds 20% padding to the right of the longest bar so value labels are never clipped.

6. ERROR HANDLING

6.1 Input Validation

Thec urrentimplementation performs implicit validation through Pandas' native exceptions. The following error conditions are handled or documented:

Error Condition	Pandas Behaviour	Recommended Guard
CSV file not found	FileNotFoundError raised at pd.read_csv()	Wrap in try/except FileNotFoundError; print descriptive message.
Missing required column	KeyError raised at first groupby call	Validate df.columns contains all 6 expected column names immediately after load.
Non-numeric Cost_USD values	TypeError raised at .sum()	Apply pd.to_numeric(df['Cost_USD'], errors='coerce') and check for resulting NaN values.
Unknown month name in data	NaN row created after reindex()	Assert set(df['Month'].unique()).issubset(set(month_order)) before aggregation.
Empty DataFrame	.sum() returns 0 Series; pie chart raises ValueError on zero-sum data	Check len(df) > 0 immediately after load.

6.2 Runtime Guards

- **WarningSuppression:** warnings.filterwarnings('ignore') suppresses matplotlib backend warnings and pandas dtype inference warnings that are non-critical for this analysis.
- **bbox_inches Guard:** All savefig() calls use bbox_inches='tight' to prevent legend and label clipping in saved chart files.
- **Context Manager (Excel):** Using pd.ExcelWriter as a context manager ensures the workbook file handle is always properly closed and flushed even if an exception occurs during sheet writing.

7. CONFIGURATION AND CONSTANTS

The following constants and configuration values are defined at pipeline initialisation and govern behaviour throughout all subsequent cells.

Constant / Setting	Value	Purpose
plt.rcParams['figure.dpi']	100	Screen rendering resolution for notebook display
savefig dpi	120	Higher resolution for saved PNG chart artefacts
display.float_format	'{:.2f}'.format	Two-decimal precision for DataFrame print output
month_order	List of 12 month name strings	Canonical calendar ordering for reindex operations
Service colour palette	['#2196F3','#4CAF50','#FF9800','#E91E63','#9C27B0']	Consistent 5-colour mapping for service types across all charts
Month colour palette	['#FF6B6B','#4ECDC4','#45B7D1','#96CEB4','#FFEAA7','#DDA0DD']	6-colour palette for monthly pie chart (repeated for 12 months)
Dept. colour palette	['#667eea','#764ba2','#f093fb','#4facfe','#00f2fe']	5-colour mapping for department bar chart
Excel engine	'openpyxl'	Backend engine for pd.ExcelWriter — required for .xlsx format

8. CONCLUSION

This Low-Level Design document defines a complete, production-ready analytical pipeline for cloud cost intelligence applied to smart city infrastructure. By precisely specifying the input data contracts, intermediate variable structures, function-level logic, chart rendering parameters, and error handling strategies, this document eliminates architectural ambiguity and ensures the resulting implementation is both reproducible and maintainable.

The pipeline's core strengths — the calendar-correct month reindex, the closure-based autopct lambda, the shared master DataFrame design, and the context-manager-protected Excel export — collectively ensure accurate, consistent analytical outputs from a minimal and clean codebase. Furthermore, the modular architecture described in this document allows straightforward extension to accommodate additional smart city departments, cloud service categories, or interactive dashboard deployment without restructuring the analytical core.

By following the specifications laid out in this LLD, the resulting implementation will successfully deliver actionable cloud cost intelligence to smart city administrators, enabling data-driven infrastructure spending decisions across all five operational departments.